DISCRETE MATHEMATICS

1. a) For each of the following statements, state whether it is true or false and provide a supporting proof.

   i) $10n^2 + 2n^3 = O(n^3)$ [3]

   Answer:
   True. $(10n^2 + 2n^3)/n^3 = 2 + 10/n \leq 12$ *for* $n \geq 1$.

   ii) $an + b2^n = O(2^n)$ for any $a \geq 0$, $b \geq 0$. [3]

   Answer:
   True. $(an + b2^n)/2^n = an/2^n + b \leq a + b/2 + 1 = c$ *for* $n \geq 1$ *(and this satisfies* $c > 0$*).*

   iii) $an^3 + bn^n = \Theta(n^n)$ for any $a \geq 0$, $b \geq 0$. [3]

   Answer:
   False. If $a = 1$ *and* $b = 0$ *then* $(an^3 + bn^n)/n^n = n^3/n^n \to 0$ *and* $n \to \infty$ *and therefore* $an^3 + bn^n = o(n^n)$ *and so cannot be* $\Theta(n^n)$.

   b) Give a tight bound for each of the following recurrence relations, or explain why it is not possible to do so.

   i) $T(n) = 7T(n/7) + n$. [2]

   Answer:
   $\log_b a = \log_7 7 = 1 = d$ *so* $T(n) = O(n^d \log n) = O(n \log n)$.

   ii) $T(n) = 9T(n/3) + \frac{n(n+1)}{2}$. [2]

   Answer:
   $\log_b a = \log_3 9 = 2 = d$ *so* $T(n) = O(n^d \log n) = O(n^2 \log n)$.

   iii) $T(n) = 3T(n/2) + kn^2$, where $k > 0$. [2]

   Answer:
   $\log_b a = \log_2 3 < \log_2 4 = 2 = d$ *so* $T(n) = O(n^d) = O(n^2)$.

2. Consider the problem of searching for a value x in an unsorted array A, and returning either an index i so that $A[i] = x$, or -1 if x is not in A.

   a) What is the best achievable time complexity for unsorted search on a single threaded architecture? [2]

   Answer:
   $O(n)$ *because in the worst case every element of A needs to be checked.*

   b) Write pseudocode for a multi-threaded version using divide and conquer. Express your answer using the keywords `spawn` and `sync`. As a guideline, your answer should not need more than 15 lines of code. [4]

   Answer:
   The following pseudocode works:

```
def psearch(arr, x):
```

Answers

```
n = len(arr)
if n<=1:
    return search(arr, x)
spawn L = psearch(arr[0 to n/2-1], x)
spawn R = psearch(arr[n/2 to n-1], x)
sync
if L>=0 return L
if R>=0 return R+n/2
return -1
```

c) Compute the work $T_1(n)$, span $T_\infty(n)$ and parallelism $T_1(n)/T_\infty(n)$ of your algorithm, carefully justifying your answers. [4]

Answer:
The work is the same as the single threaded case, $T_1(n) = n$. The span satisfies $T_\infty(n) = \Theta(1) + T_\infty(n/2)$ because the work done in each call is fixed $\Theta(1)$ and the parallel calls execute on an array of half the size in parallel. This gives a result of $T_\infty(n) = \Theta(\log n)$. The paralellism is therefore $\Theta(n/\log n)$.

Answers

3. Countup is a game of finding the shortest way to get to a target number n, starting at 0, with a fixed set of operations F. For example, if the target is $n = 6$ and the operations in F are A_1="add 1" and M_3="multiply by 3" then you could get to 6 by doing $A_1A_1A_1A_1A_1A_1 = 0+1+1+1+1+1+1$, but it would be shorter to do $A_1A_1M_3 = (0+1+1)\times 3$. Your goal is to efficiently cheat at this game using the power of algorithms.

Use the notation $\mathbb{N}^+ = \{1,2,3,\ldots\}$ for the set of positive integers. You may assume the following: $n \in \mathbb{N}^+$ is a positive integer; all operations $f \in F$ are of the form $f = A_i$ ="add i" or $f = M_j$ ="multiply by j", where $i, j \in \mathbb{N}^+$; in every case, $A_1 \in F$, guaranteeing that there is a solution of length at most n; the number of operations $|F|$ is fixed and does not depend on n.

a) A brute force solution to this problem is to iterate through every sequence of operations of length up to n, check if they match the target n, and return the shortest sequence. Compute the time complexity of this algorithm as a function of n. [4]

Answer:
There are $|F|^n$ sequences of operations in F of length n. Computing partial sequences as you go, this gives time complexity $O(|F|^n)$. Other answers that specify a particular data structure and have a higher time complexity are acceptable.

b) We can write a dynamic program to compute the length of the shortest solution as follows. Write f^{-1} to be the inverse function of f, so $A_i^{-1}(x) = x - i$, etc. Consider the problem of starting at n and finding the shortest sequence of inverse functions that arrives at 0. For the example of $n = 6$ and $F = \{A_1, M_3\}$ we get that $0 = A_1^{-1}(A_1^{-1}(M_3^{-1}(6)))$ which lets us reconstruct the solution $A_1A_1M_3$ to the original problem.

i) Write an equation for $C(n)$, the length of the shortest such sequence, in terms of $C(0), C(1), \ldots, C(n-1)$.

Hint: consider a series of choices taking you closer to 0. Make sure to check which choices are valid, and to consider the base case. [6]

Answer:

$$C(n) = \begin{cases} 0 & n = 0 \\ 1 + \min_{f \in F : f^{-1}(n) \in \mathbb{N}} C(f^{-1}(n)) & n > 0 \end{cases}$$

ii) Without writing pseudocode, compute the time complexity of this algorithm. [4]

Answer:
F has fixed size, so the min operation is $O(1)$ and we need to compute n values of C so the total cost is $O(n)$.

iii) Write pseudocode for this algorithm. As a guideline, it should not take more than 20 lines of code. You may write f^{-1} or `f.inverse` as the inverse of an operation. [6]

Answer:

Answers

The following Python code works:

```
def DP(n, ops):
    memo = {0: 0}
    def C(n):
        if n not in memo:
            best = n+1
            for f in ops:
                k = f.inverse(n)
                if k!=int(k) or k<0:
                    continue
                if 1+C(k)<best:
                    best = 1+C(k)
            memo[n] = best
        return memo[n]
    return C(n)
```

c) Find the set of all solutions with the shortest length in the case $n = 11$, $F = \{A_1, A_2, M_3, M_4\}$.

Hint: use the same basic structure of your dynamic program, but modify it to find all solutions; or alternativel use the brute-force algorithm. [5]

Answer:
The idea is to keep a table of each integer $n = 1$ to 11, where each row of the table has the total count of operations to get to 0 from there, as well as the set of equally good first operations. Then, we reconstruct the full set by taking all paths through this table.

n	$C(n)$	*Next operations*
0	0	-
1	1	A_1
2	1	A_2
3	2	A_1, A_2, M_3
4	2	A_2, M_4
5	3	A_1, A_2
6	2	M_3
7	3	A_1
8	2	M_4
9	3	A_1, M_3
10	3	A_2
11	4	A_1, A_2

So from this we can see that the best solution has length 4, and the following are possible. We either start with A_1 or A_2. If we start with A_1 it takes us to 10, so then we need A_2, which takes us to 8, and so on. We get five solutions in this way: $A_2M_4A_2A_1$, $A_2M_4A_1A_2$, $A_2A_1M_3A_2$, $A_1A_2M_3A_2$, $A_1M_3M_3A_2$.

The following Python code implements this, but this is not required for the answer to this question:

```
def DP_all(n, ops):
    memo_count = {0: 0}
    memo_next = {0: []}
    def C(n):
```

Answers

```
        if n not in memo_count:
            best_choices = []
            best_count = n+1
            for f in ops:
                k = f.inverse(n)
                if k!=int(k) or k<0:
                    continue
                if 1+C(k)<best_count:
                    best_count = 1+C(k)
                    best_choices = [f]
                elif 1+C(k)==best_count:
                    best_choices.append(f)
            memo_count[n] = best_count
            memo_next[n] = best_choices
        return memo_count[n]
    C(n)
    def reconstruct(n):
        if n==0:
            return [[]]
        solutions = []
        for f in memo_next[n]:
            k = f.inverse(n)
            for sol in reconstruct(k):
                solutions.append(sol+[f])
        return solutions
    return reconstruct(n)
```

Answers